Empezamos con la Tanda 1: Esqueleto + Cabecera

Crea el archivo src/language/parser.py y pega este código:

Python

```
from typing import List, Optional
from src.language.tokens import Token, TokenType
from src.language.ast_nodes import CompiledProgram
from src.language.errors import CompileError


# Las 5 claves obligatorias de la cabecera (según el enunciado)
REQUIRED_HEADER_KEYS = {"creature", "faction", "health", "vision", "lifespan"}


class Parser:
    """
    Toma la lista de tokens generada por el Lexer y construye un CompiledProgram.
    Implementa un análisis por descenso recursivo.
    """

    def __init__(self, tokens: List[Token]):
        self.tokens = tokens
        self.current = 0
        self.program = CompiledProgram()

    # ========================================================================
    # PUNTO DE ENTRADA
    # ========================================================================

    def parse(self) -> CompiledProgram:
        """Compila el archivo entero y devuelve un CompiledProgram listo."""
```

```
    self._skip_newlines()
    self._parse_header()
    # TANDA 3 (más adelante): self._parse_body()
    return self.program

# ============================================================================
# CABECERA
# ============================================================================

def _parse_header(self) -> None:
    """
    Lee las 5 líneas de la cabecera. La primera debe ser 'creature Name'.
    Las otras 4 pueden estar en cualquier orden.
    Se detiene al encontrar la etiqueta 'start:'.
    """
    seen_keys = set()

    # Verificar que la PRIMERA línea no vacía sea 'creature Name'
    if not self._check_identifier("creature"):
        raise CompileError(
            "La primera línea del archivo debe ser 'creature NombreEspecie'",
            self._peek().line
        )

    # Leer líneas de cabecera hasta encontrar 'start:'
    while not self._is_at_end():
        # Si detectamos el comienzo del cuerpo, terminamos la cabecera
        if self._is_start_label():
            break
```

```
        key_token = self._advance()  # Consumimos el identificador (ej. 'creature', 'health')

        if key_token.type != TokenType.IDENTIFIER:
            raise CompileError(
                f"Se esperaba una clave de cabecera (creature, faction, health, vision, lifespan), "
                f"pero se encontró: '{key_token.value}'",
                key_token.line
            )

        key = key_token.value

        if key not in REQUIRED_HEADER_KEYS:
            raise CompileError(
                f"Clave de cabecera desconocida: '{key}'. "
                f"Se esperaba una de: {REQUIRED_HEADER_KEYS}",
                key_token.line
            )

        if key in seen_keys:
            raise CompileError(
                f"La clave de cabecera '{key}' está declarada más de una vez",
                key_token.line
            )

        seen_keys.add(key)
        self._parse_header_value(key, key_token.line)
        self._consume_newline(f"Se esperaba salto de línea tras la declaración de '{key}'")
        self._skip_newlines()

    # Al terminar, comprobar que se hayan leído las 5 claves
```

```
    missing = REQUIRED_HEADER_KEYS - seen_keys
    if missing:
        raise CompileError(
            f"Faltan claves obligatorias en la cabecera: {missing}",
            self._peek().line
        )

def _parse_header_value(self, key: str, line: int) -> None:
    """Lee el valor asociado a una clave de cabecera y valida su tipo."""
    value_token = self._advance()

    # creature y faction esperan un nombre (identificador)
    if key in ("creature", "faction"):
        if value_token.type != TokenType.IDENTIFIER:
            raise CompileError(
                f"'{key}' requiere un nombre (identificador), no '{value_token.value}'",
                line
            )
        if key == "creature":
            self.program.creature_name = value_token.value
        else:
            self.program.faction = value_token.value

    # health, vision y lifespan esperan un número entero positivo
    elif key in ("health", "vision", "lifespan"):
        if value_token.type != TokenType.NUMBER:
            raise CompileError(
                f"'{key}' requiere un número entero, no '{value_token.value}'",
                line
            )
```

```
        num = value_token.value

        if key == "health" and num <= 0:
            raise CompileError("'health' debe ser mayor que 0", line)
        if key == "vision" and num < 1:
            raise CompileError("'vision' debe ser mayor o igual que 1", line)
        if key == "lifespan" and num <= 0:
            raise CompileError("'lifespan' debe ser mayor que 0", line)

        if key == "health":
            self.program.health = num
        elif key == "vision":
            self.program.vision = num
        else:
            self.program.lifespan = num

# ============================================================================
# UTILIDADES DE NAVEGACIÓN DE TOKENS
# ============================================================================

def _peek(self, offset: int = 0) -> Token:
    """Mira el token en la posición actual + offset sin consumirlo."""
    pos = self.current + offset
    if pos >= len(self.tokens):
        return self.tokens[-1]  # EOF
    return self.tokens[pos]

def _advance(self) -> Token:
    """Consume el token actual y avanza al siguiente."""
    token = self.tokens[self.current]
```

```
    if not self._is_at_end():
        self.current += 1
    return token

def _is_at_end(self) -> bool:
    return self._peek().type == TokenType.EOF

def _check(self, token_type: TokenType) -> bool:
    return self._peek().type == token_type

def _check_identifier(self, name: str) -> bool:
    """Comprueba si el token actual es un IDENTIFIER con ese nombre exacto."""
    return self._check(TokenType.IDENTIFIER) and self._peek().value == name

def _is_start_label(self) -> bool:
    """Detecta el patrón 'start' ':' para saber que empieza el cuerpo."""
    return (
        self._check_identifier("start")
        and self._peek(1).type == TokenType.COLON
    )

def _skip_newlines(self) -> None:
    """Salta todos los saltos de línea consecutivos (líneas en blanco)."""
    while self._check(TokenType.NEWLINE):
        self._advance()

def _consume_newline(self, error_msg: str) -> None:
    """Espera un salto de línea. Si no lo encuentra, error."""
    if not self._check(TokenType.NEWLINE) and not self._is_at_end():
        raise CompileError(error_msg, self._peek().line)
```

```
        if self._check(TokenType.NEWLINE):
            self._advance()
```

Ahora vamos a probarlo

Crea el archivo tests/test_parser.py:

Python

```
from src.language.lexer import Lexer
from src.language.parser import Parser


# Ejemplo mínimo con solo la cabecera (el cuerpo lo añadimos en la Tanda 3)
sample_code = """# uruk.ins
creature Uruk
faction isengard
health 80
vision 6
lifespan 400

start:
"""


def test_header_parsing():
    print("=== Tokenizando ===")
    lexer = Lexer(sample_code)
    tokens = lexer.tokenize()
    print(f"Tokens generados: {len(tokens)}\n")

    print("=== Parseando ===")
    parser = Parser(tokens)
```

```
    program = parser.parse()

    print("=== Resultado ===")
    print(program)



def test_header_missing_key():
    """Verifica que el parser detecte una cabecera incompleta."""
    bad_code = """creature Uruk
faction isengard
health 80
start:
"""
    print("\n=== Prueba: cabecera incompleta ===")
    try:
        tokens = Lexer(bad_code).tokenize()
        Parser(tokens).parse()
        print("ERROR: Debía haber lanzado excepción")
    except Exception as e:
        print(f"OK - Se detectó el error: {e}")



def test_header_wrong_first_line():
    """Verifica que el parser detecte un archivo que no empiece por 'creature'."""
    bad_code = """faction isengard
creature Uruk
health 80
vision 6
lifespan 400
start:
```

```
"""
    print("\n=== Prueba: primera línea incorrecta ===")
    try:
        tokens = Lexer(bad_code).tokenize()
        Parser(tokens).parse()
        print("ERROR: Debía haber lanzado excepción")
    except Exception as e:
        print(f"OK - Se detectó el error: {e}")



if __name__ == "__main__":
    test_header_parsing()
    test_header_missing_key()
    test_header_wrong_first_line()
```